Multi-Agent AI Systems
Workshop

Building AI Agents That Actually Work

Day 1: Concepts + Start Building → Day 2: Milestones 3-5 + Demos

OVERVIEW

Your Learning Journey

AI / GenAI & LLMs → LLM APIs & Tokens → Tool & Function Call → ReAct & Reflexion → Single Agent → Multi-Agent Architecture

Track A: Document Intelligence Hub

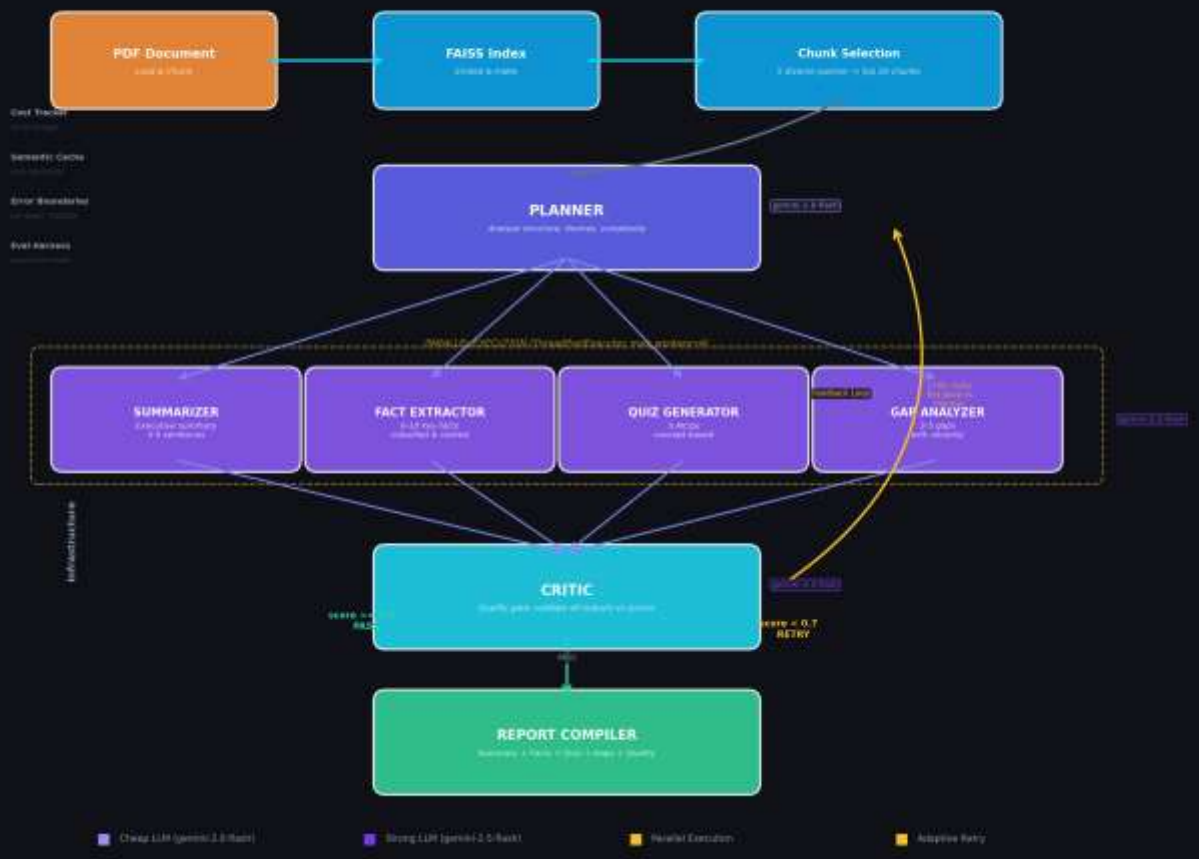
Planner → Summarizer, Facts, Quiz, Gaps (parallel) → Critic → Retry loop
5 milestones: sequential → parallel → cost → eval → adaptive retry

Track B: Multi-Agent Debate System

Planner → Researchers → Debaters → Judge → Cross-exam → Synthesizer
5 milestones: sequential → parallel → cross-exam → cost → tournament

Track A: Document Intelligence Hub

7 Cooperative Agents | 4 Parallel Workers | Adaptive Retry Loop



Track B: Multi-Agent Debate System

7 Adversarial Agents | Parallel FOR vs AGAINST | Multi-Round Debate

Track C: Document Q&A with Fact-Check

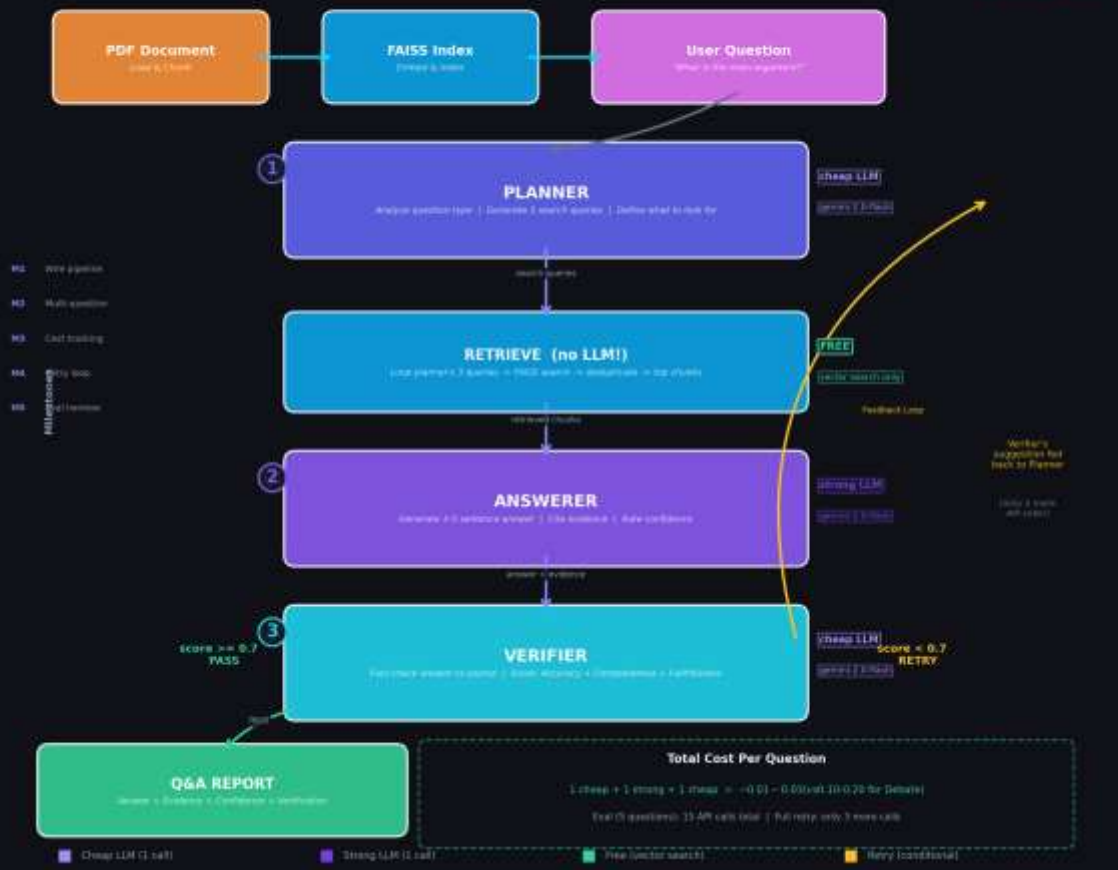
3 Agents | 3 LLM Calls | Rate-Limit Friendly | ~$0.01-0.03 per question

API Calls Comparison

Debate System: 5-8 calls

Intelligence Hub: 6 calls

Q&A Fact-Check: 3 calls



Shared Infrastructure: helpers.py

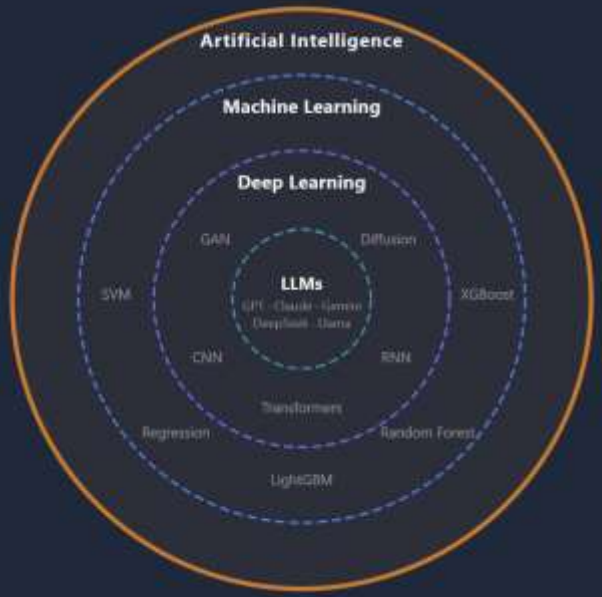
Building blocks used by BOTH tracks | Multi-provider LLM abstraction

All functions return structured dicts → easy to chain agents together

CONCEPTS

The AI Landscape

AI, ML, Deep Learning, and LLMs are nested layers - click any to explore



Artificial Intelligence
Machines that perform tasks requiring human intelligence →

Machine Learning
Algorithms that learn patterns from data

Deep Learning
Neural networks with many layers

Large Language Models
Massive models trained on internet-scale text

Click a layer or circle to explore

CONCEPTS

The Evolution of AI

From hand-written rules to machines that learn - how AI approaches evolved over 70 years

1950s–80s

Rule-Based AI

Hand-coded if/else rules

Chess: "if queen threatened, move queen"

→

1980s–90s

Expert Systems

Encoded human expert knowledge

Medical: 500+ rules for diagnosing diseases

→

2000s+

Machine Learning

Algorithms learn from data

Spam filter: learns from 1M+ emails

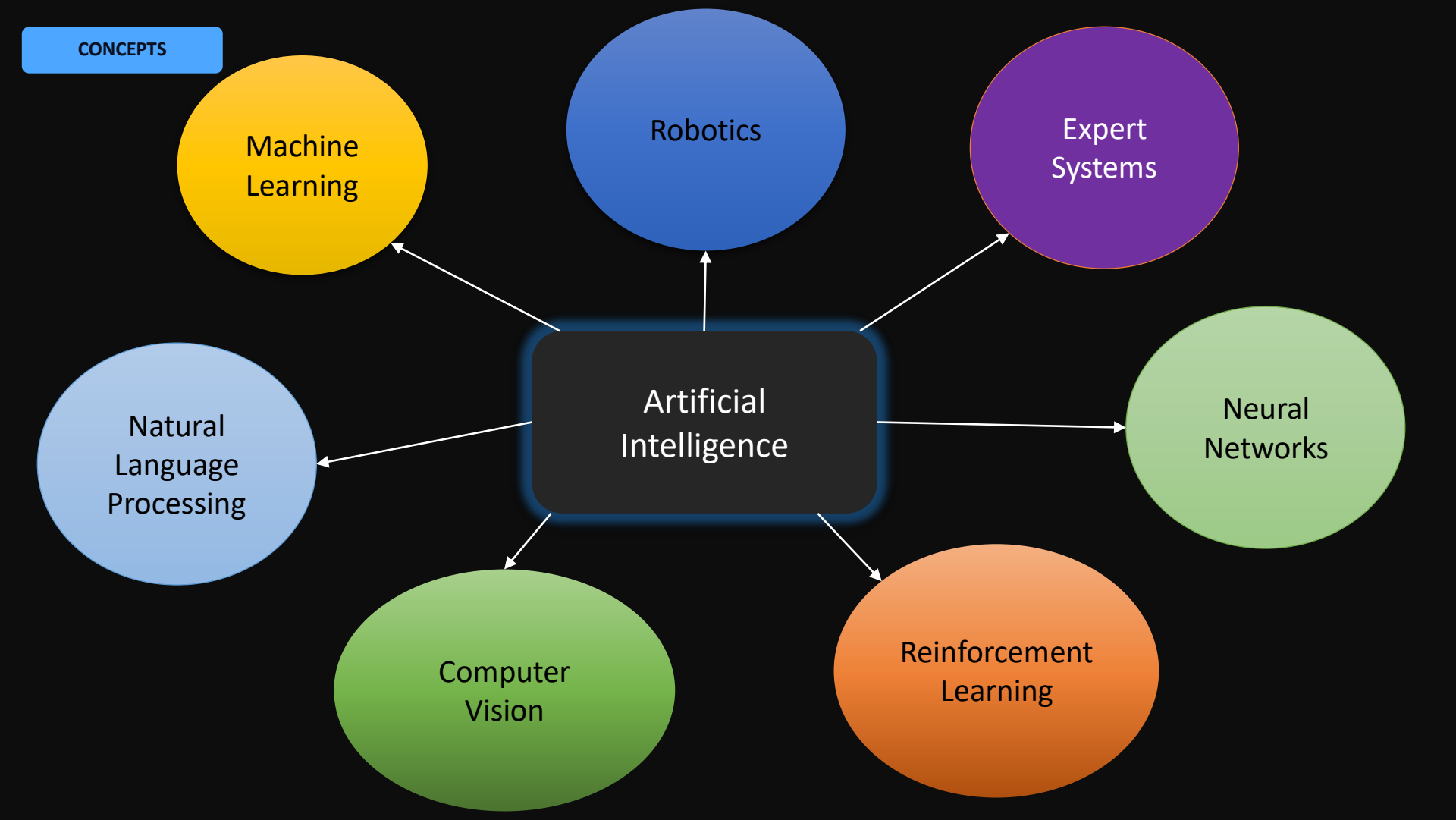
→

2010s+

Deep Learning & LLMs

Neural nets + massive data

GPT/Claude: trained on internet-scale text


CONCEPTS
Machine Learning
Robotics
Expert Systems
Natural Language Processing
Artificial Intelligence
Neural Networks
Computer Vision
Reinforcement Learning

CONCEPTS

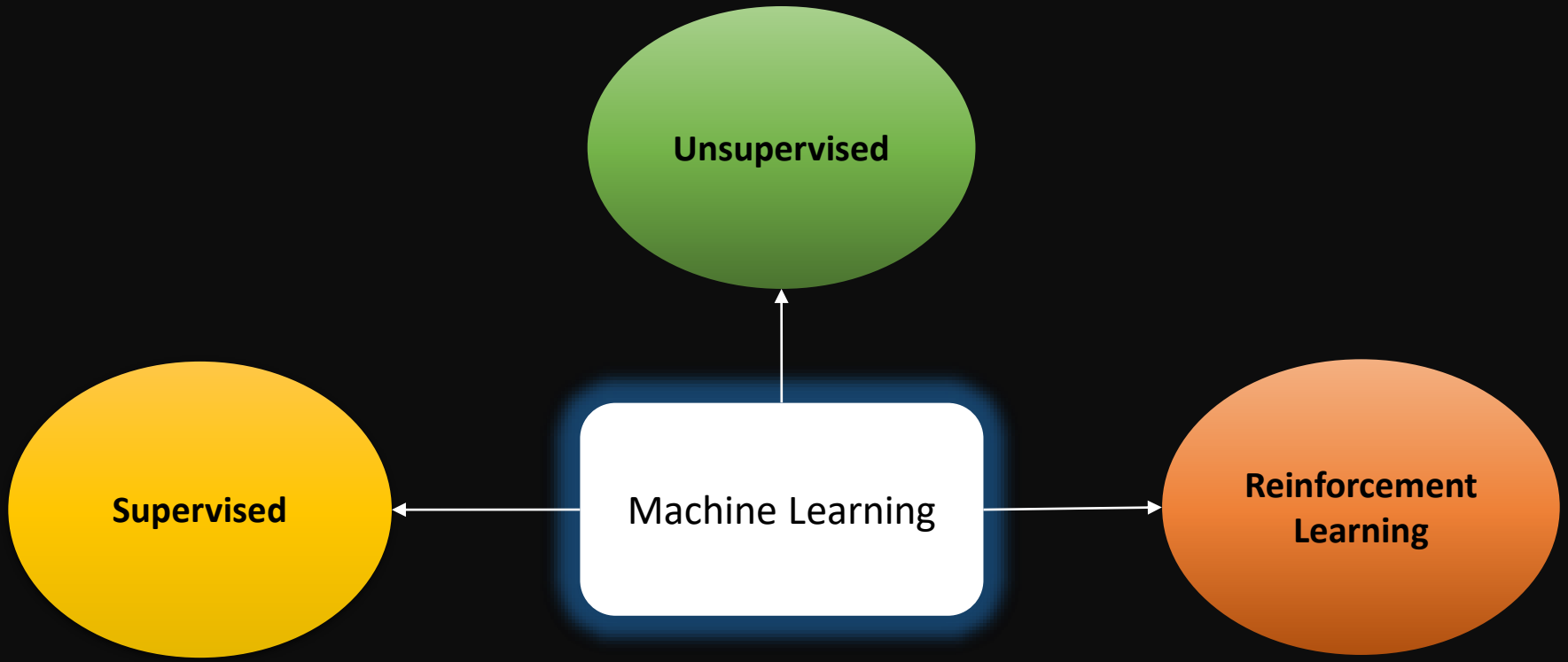
What is Machine Learning?

Machine Learning is a way to teach computers by showing them examples instead of writing rules. Instead of telling a computer exactly what to do, you give it data and let it discover the patterns on its own.

Traditional	vs	Machine Learning
You write the rules		Computer learns from data

Why it matters: Some problems have too many rules to write by hand - recognizing faces, understanding speech, predicting weather. ML handles complexity that traditional programming cannot.

CONCEPTS


Unsupervised
Supervised
Machine Learning
Reinforcement Learning

CONCEPTS

What is Deep Learning?

Deep Learning is ML using neural networks with many layers. Inspired by the brain, these networks learn hierarchical representations - from simple features (edges) to complex concepts (faces).

Why "Deep"? Each layer learns more abstract features:

Pixels (Raw input) → Edges (Layer 1) → Textures (Layer 2) → Parts (Layer 3) → Object! (Output)

Traditional ML needs you to define features manually. Deep networks discover them on their own!

More Data

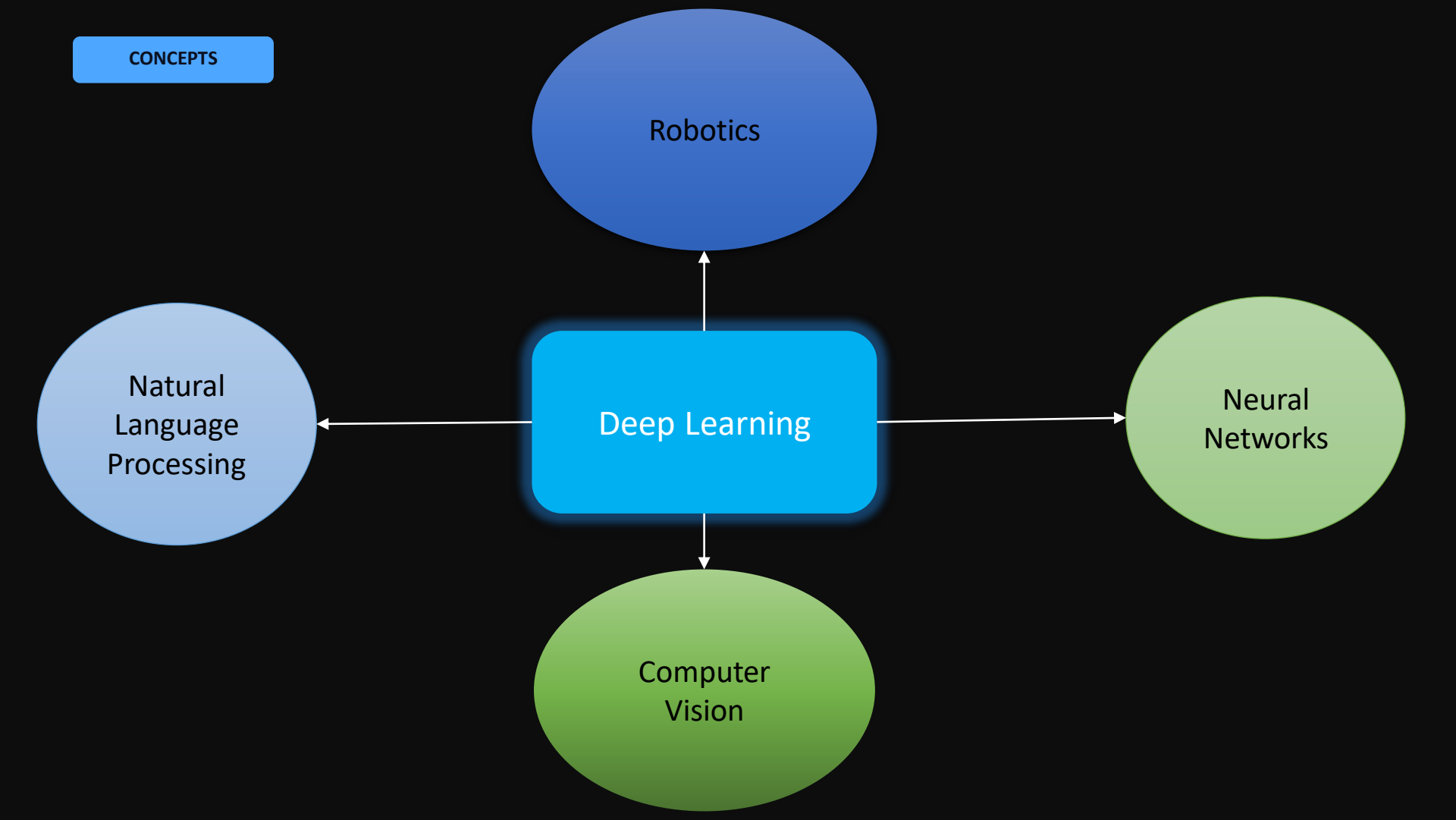
DL excels with massive datasets (millions of examples)

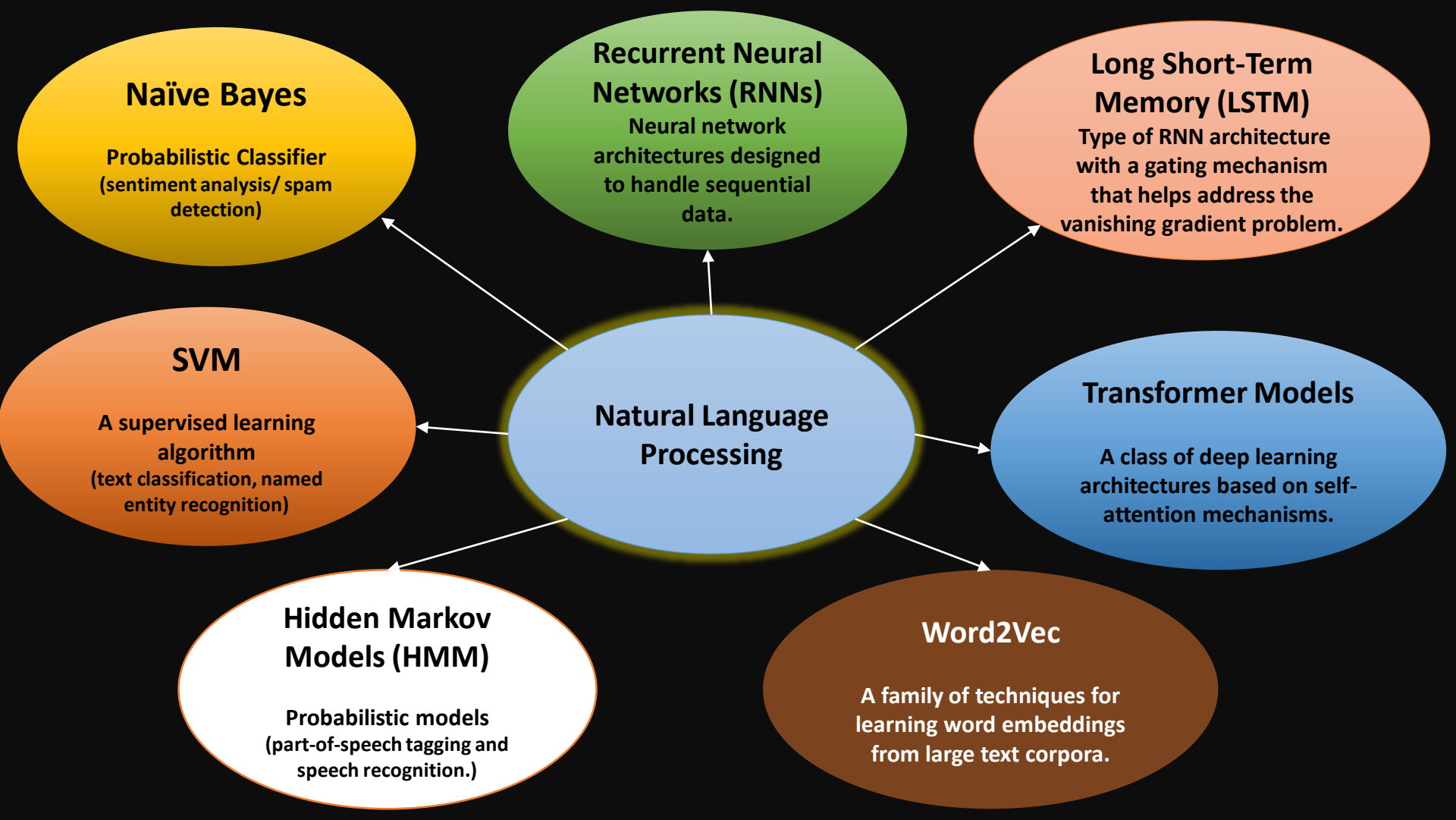
Unstructured Data

Images, audio, text - DL handles it all

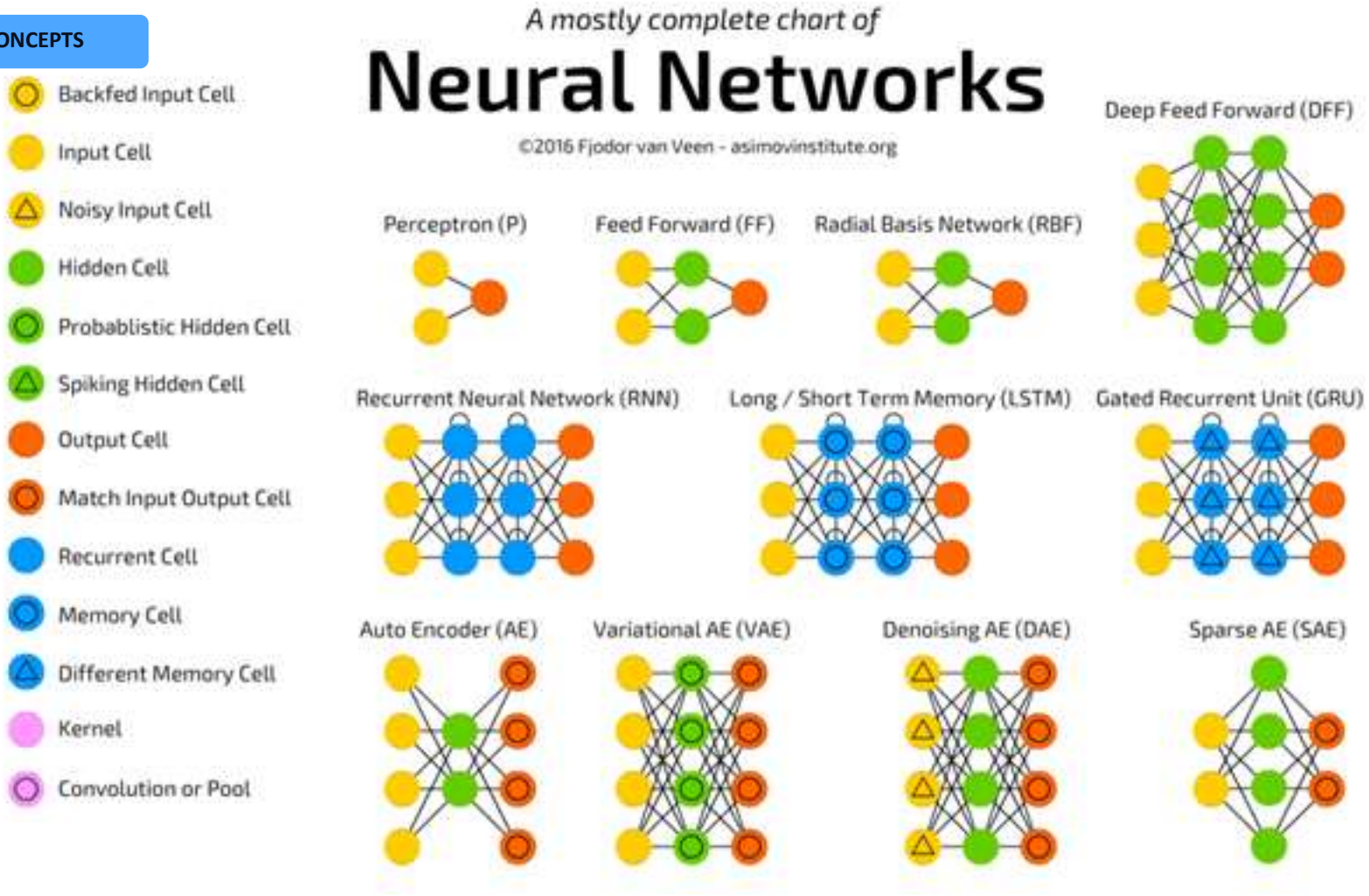
GPU Power

Parallel computation on graphics cards


CONCEPTS
Robotics
Natural Language Processing
Deep Learning
Neural Networks
Computer Vision

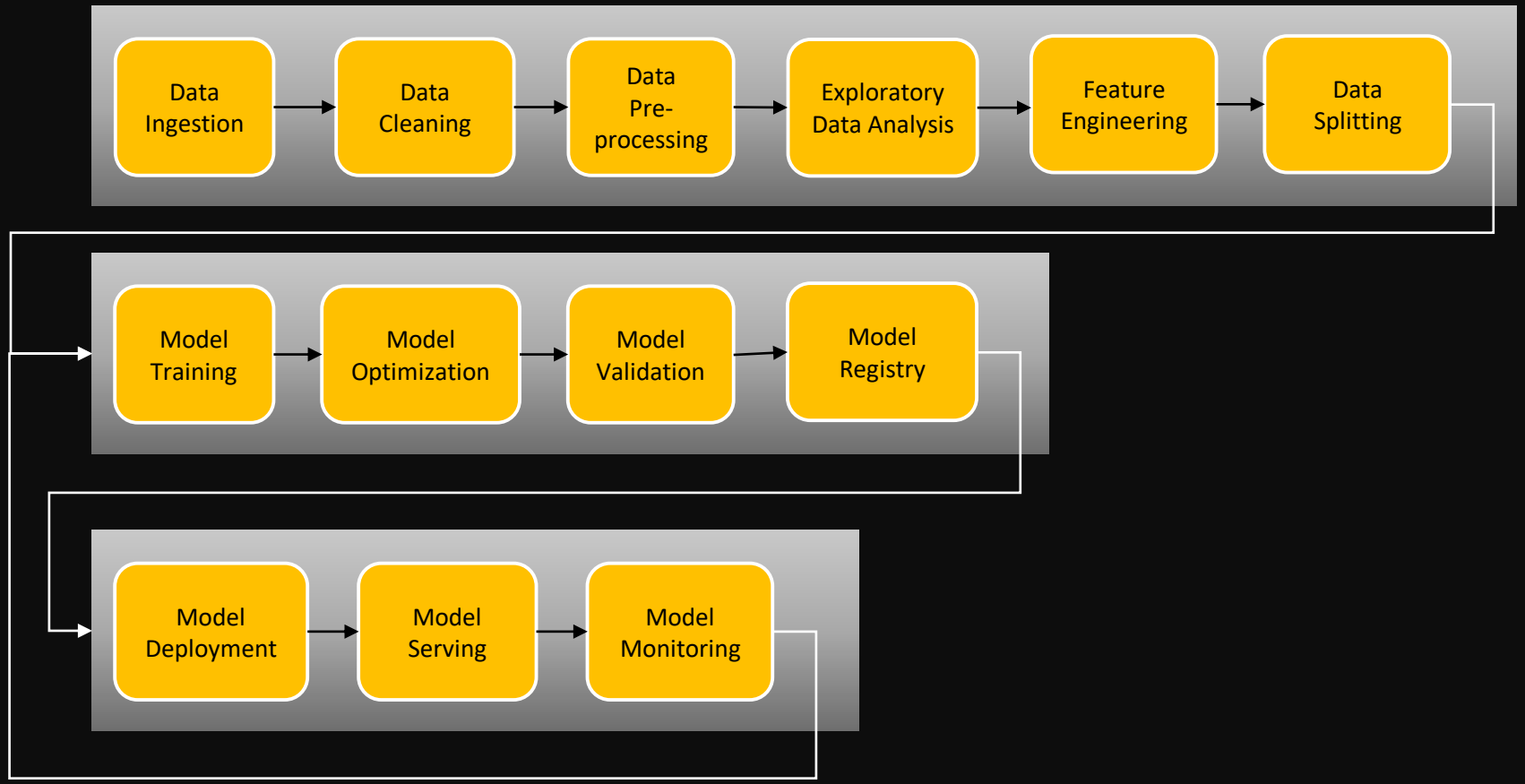

Naïve Bayes
Probabilistic Classifier
(sentiment analysis/ spam detection)
Recurrent Neural Networks (RNNs)
Neural network architectures designed to handle sequential data.
Long Short-Term Memory (LSTM)
Type of RNN architecture with a gating mechanism that helps address the vanishing gradient problem.
SVM
A supervised learning algorithm
(text classification, named entity recognition)
Natural Language Processing
Transformer Models
A class of deep learning architectures based on self-attention mechanisms.
Hidden Markov Models (HMM)
Probabilistic models
(part-of-speech tagging and speech recognition.)
Word2Vec
A family of techniques for learning word embeddings from large text corpora.

CONCEPTS


A mostly complete chart of
Neural Networks
©2016 Fjodor van Veen - asimovinstitute.org
Backfed Input Cell
Input Cell
Noisy Input Cell
Hidden Cell
Probablistic Hidden Cell
Spiking Hidden Cell
Output Cell
Match Input Output Cell
Recurrent Cell
Memory Cell
Different Memory Cell
Kernel
Convolution or Pool
Perceptron (P)
Feed Forward (FF)
Radial Basis Network (RBF)
Deep Feed Forward (DFF)
Recurrent Neural Network (RNN)
Long / Short Term Memory (LSTM)
Gated Recurrent Unit (GRU)
Auto Encoder (AE)
Variational AE (VAE)
Denoising AE (DAE)
Sparse AE (SAE)

CONCEPTS

Machine Learning Pipeline



CONCEPTS

What are Large Language Models?

LLMs are massive neural networks trained to predict the next token in a sequence. They're large because they have hundreds of billions (or even trillions) of parameters and are trained on internet-scale datasets containing trillions of tokens from books, websites, code repositories, scientific papers, and more.

Today's Frontier Models

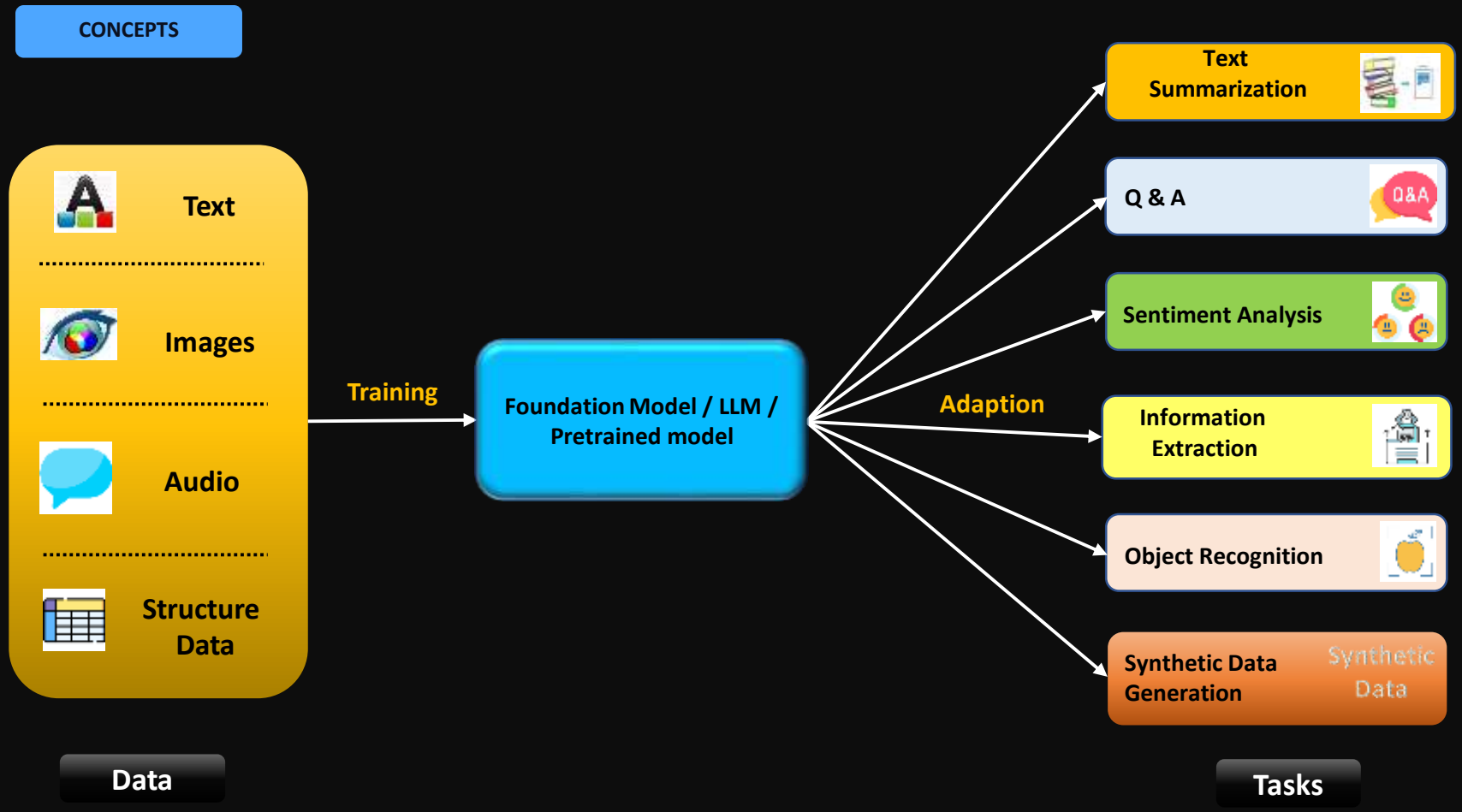
GPT-5.2	Claude Opus 4.6	Gemini 3 Pro	Grok 4.1	DeepSeek V3.2	Llama 4
OpenAI	Anthropic	Google	xAI	DeepSeek	Meta

1T+ Parameters × 15T+ Training tokens × 10K+ GPUs = Intelligence

Internet-Scale Data
Trillions of tokens

Next Token Prediction
Simple, profound

Emergent Abilities
Reasoning at scale

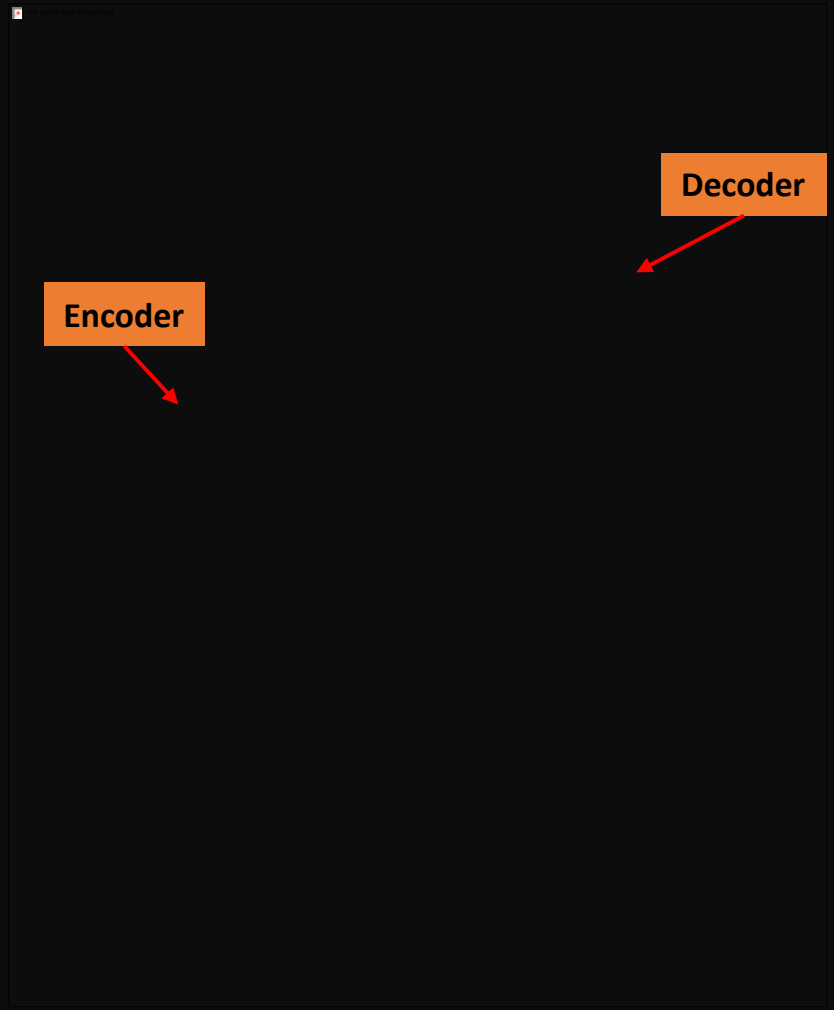

CONCEPTS
Text
Images
Audio
Structure Data
Data
Training
Foundation Model / LLM / Pretrained model
Adaption
Text Summarization
Q & A
Q&A
Sentiment Analysis
Information Extraction
Object Recognition
Synthetic Data Generation
Synthetic Data
Tasks

CONCEPTS

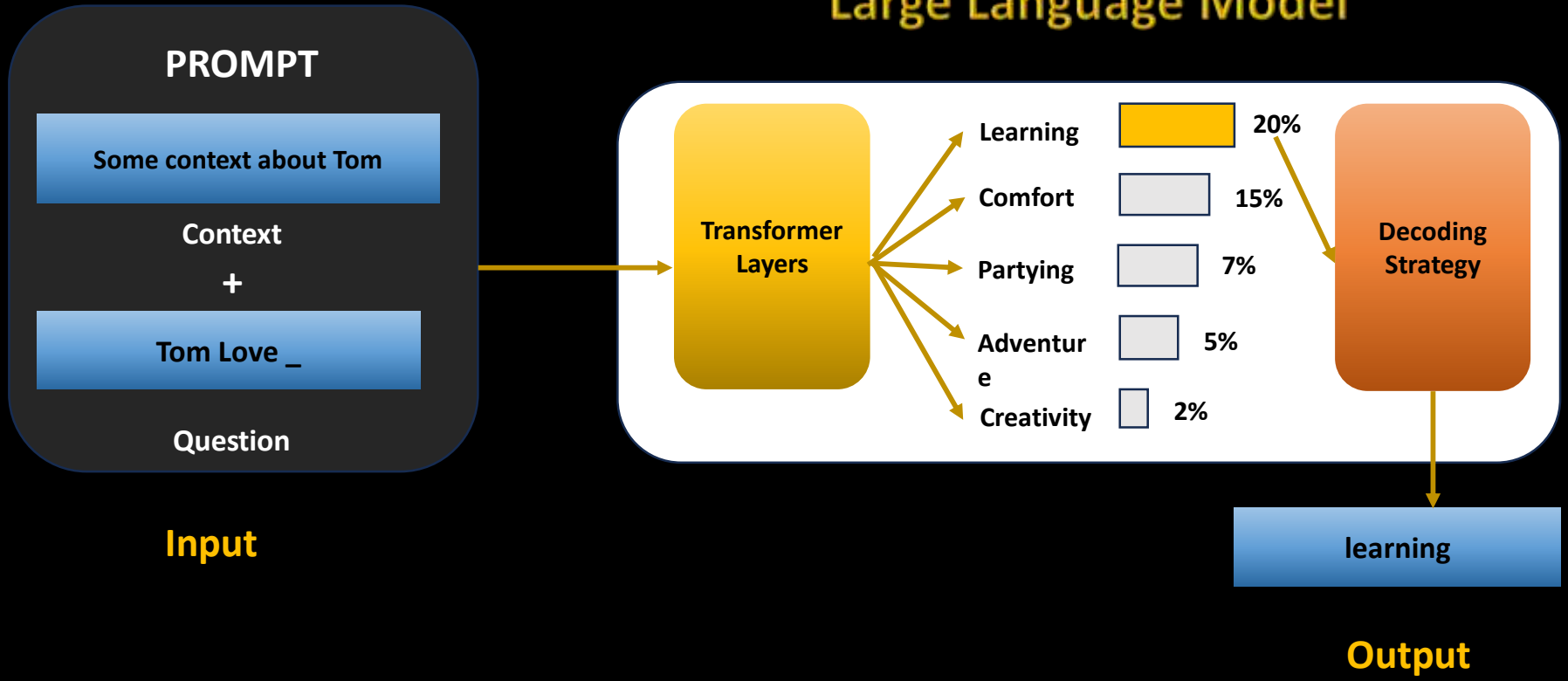
Transformer Architecture

Backbone of

Large Language Model



CONCEPTS


PROMPT
Some context about Tom
Context
+
Tom Love _
Question
Input
Large Language Model
Transformer Layers
Learning
20%
Comfort
15%
Partying
7%
Adventure
5%
Creativity
2%
Decoding Strategy
learning
Output

CONCEPTS

What is Generative AI?

AI that creates new content - text, images, code, music - rather than just classifying.

Traditional AI

Classification:
"Is this email spam?"

Prediction:
"What will the stock price be?"

Detection:
"Is there a cat in this image?"

Generative AI

Text Generation:
"Write a summary of this document"

Code Generation:
"Build a REST API for user login"

Reasoning:
"Analyze this data and explain trends"

Key Insight: GenAI doesn't just answer questions — it produces novel content. LLMs are the engine behind text-based GenAI.

CONCEPTS

How Large Language Models Work

An LLM predicts the next token - one word at a time - based on everything before it.

Training Data

Trillions of words from
books, web, code, papers

→

Transformer Model

Self-attention mechanism
learns language patterns

→

Next Token Prediction

"The cat sat on the ___"
Model predicts: "mat" (92%)

Key LLM Families

GPT-4o / GPT-4o-mini	Claude Sonnet / Haiku	Gemini 2.5	Llama 3.x
OpenAI	Anthropic	Google	Meta (Open)

CONCEPTS

Tokens, Context Windows & Limitations

What is a Token?

1 token ≈ ¾ of a word
"Hello world" = 2 tokens
"Unbelievable" = 3 tokens (un + believ + able)

Context Window

Max tokens the model sees at once
GPT-4o: 128K tokens (~96K words)
Claude: 200K tokens (~150K words)

Hallucination

LLMs confidently generate false info

No Real-Time Knowledge

Training data has a cutoff date

Lost in the Middle

LLMs ignore info in center of long prompts

These limitations are exactly why we need RAG (retrieval) and multi-agent architectures.

CONCEPTS

LLM APIs: Your Interface to Intelligence

You send HTTP requests to cloud APIs and get responses back. helpers.py wraps this for you.

OpenAI API

```
from openai import OpenAI
client = OpenAI()

response = client.chat.completions.create(
    model="gpt-4o",
    messages=[
        {"role": "system", "content": "..."},
        {"role": "user", "content": "..."}
    ],
    max_tokens=500,
    temperature=0
)
print(response.choices[0].message.content)
```

Anthropic API

```
import anthropic
client = anthropic.Anthropic()

response = client.messages.create(
    model="claude-sonnet-4-20250514",
    max_tokens=500,
    system="...",
    messages=[
        {"role": "user", "content": "..."}
    ]
)
print(response.content[0].text)
```

helpers.py gives you call_llm_cheap() and call_llm_strong() — auto-picks model, handles retries & JSON parsing.

CONCEPTS

Tool Calling & Function Calling

LLMs can request actions (search, calculate, API calls). Your code executes them.

1 Define Tools

List functions with names, descriptions, params

2 LLM Chooses

Model decides which tool to call (if any)

3 You Execute

Your code runs the actual function

4 LLM Responds

Model uses the result to answer the user

In Your Workshop Code

Your agents ARE the tools. Each agent function is a specialized "tool" the pipeline calls:

Planner → decides what sub-tasks to run (like an LLM choosing tools)
Researcher/Summarizer → executes the task (like your code running a function)
Critic/Judge → validates the result (quality gate before responding)

The pipeline orchestrates them — same pattern as tool calling, just at a higher level.

CONCEPTS

ReAct: Reasoning + Acting

Think step-by-step, take actions, observe results, repeat until done.

Thought

LLM reasons about
what to do next

Action

LLM calls a tool
(search, calculate)

Observation

Tool returns result.
LLM reads it.

↻ Repeat until answer is ready

Example: "Who approved the Q3 budget and what was the amount?"

Thought: I need to search for Q3 budget approval. **Action:** search("Q3 budget approval")
Observation: "CFO approved $2.1M on Sept 15."
Thought: I have who (CFO) and amount ($2.1M). Ready to answer.

CONCEPTS

Reflexion: Learning from Mistakes

Generate → Evaluate → Reflect → Retry. This is exactly how your Critic agent works.

1 Generate

Agent produces output using available context

→

2 Evaluate

Critic scores quality against source chunks

→

3 Reflect & Retry

If score < 0.7, Planner gets feedback and pipeline re-runs

ReAct

Think → Act → Observe → Repeat
Focuses on taking the right actions
Used by: Researchers, Debaters

Reflexion

Generate → Evaluate → Reflect → Retry
Focuses on improving output quality
Used by: Critic → Planner retry loop

CONCEPTS

From Single Agent to Multi-Agent

Why one LLM call isn't enough — and how specialization solves it.

Context Overload

"Lost in the middle" — LLMs ignore facts buried in long prompts.
→ Solution: retrieve only what's needed (RAG)

Too Many Jobs

One agent doing search + writing + checking = bad at all.
→ Solution: one agent, one job (like software modules)

No Quality Check

Nobody verifies the answer. Hallucinations reach the user.
→ Solution: a Critic agent checks before output

These three problems motivate everything in your workshop code: RAG, specialized agents, Critic/Judge.

CONCEPTS

Multi-Agent Patterns (Both in Your Code)

Sequential Pipeline + Parallel Fan-out

Planner → [4 agents in parallel] → Critic

Used by: Document Intelligence Hub
Planner runs first, then Summarizer, Fact Extractor, Quiz Gen, Gap Analyzer all run in parallel via ThreadPoolExecutor, then Critic validates.

Adversarial Pipeline + Debate

Planner → [2 Researchers ||] → [2 Debaters ||] → Judge → [Cross-exam ||] → Judge R2

Used by: Debate System
Two sides argue with evidence. Judge scores. If close, cross-exam triggers round 2.

Reflexion / Retry Loop

Critic scores < 0.7 → feedback to Planner → re-run.
Hub Milestone 5. Budget-aware: skip if too costly.

Shared State Dict

Both tracks use a shared Python dict as agent memory. init_state() → agents modify → log_agent() tracks everything.

RECAP

Concept Check: Can You Explain Each?

Generative AI
AI that creates new content (text, code, images)

ReAct
Think → Act → Observe loop for multi-step reasoning

LLM
Transformer model that predicts the next token

Reflexion
Generate → Evaluate → Reflect → Retry for quality

LLM API
HTTP interface to send prompts and receive completions

Single → Multi-Agent
Specialized agents sharing state in a pipeline

Tool / Function Calling
LLM requests actions; your code executes them

Parallel Execution
ThreadPoolExecutor runs independent agents simultaneously

Now let's set up the environment and start building.

SETUP

Environment Setup

Do this together. Raise your hand if something breaks.

```
# Step 1 – Check Python (need 3.10+)
python --version

# Step 2 – Install from requirements.txt
pip install -r requirements.txt
# Installs: openai, anthropic, pymupdf,
#           faiss-cpu, pydantic, python-dotenv

# Step 3 – Set API key
cp _env.example .env
# Edit .env: add your OpenAI or Anthropic key

# Step 4 – Test everything
python helpers.py
# Should print: "All tests passed. Ready for labs."
```

Common Errors

pip not found
→ Use pip3 or python -m pip

faiss-cpu fails on Mac M1/M2
→ pip install faiss-cpu --no-binary :all:

ModuleNotFoundError: fitz
→ pip install pymupdf (not fitz)

API key not working
→ Check .env is in same folder

Allow 30 min for setup. Run `python helpers.py` — if it prints 'All tests passed', you're good.

TOOLKIT

helpers.py — Your Building Blocks

Everything is provided. You focus on the interesting part: agent orchestration.

`load_and_chunk(pdf_path)`	PDF → text → overlapping 300-word chunks
`build_index(chunks)`	Embed chunks → FAISS vector index
`search(index, chunks, query, k=5)`	Semantic search → top-k similar chunks
`call_llm_cheap() / call_llm_strong()`	Haiku/GPT-mini vs Sonnet/GPT-4o, with retry
`parse_json(text)`	Safely extract JSON from LLM output
`CostTracker(budget=0.50)`	Track tokens, cost, enforce budget limits
`EvalHarness()`	Benchmark pipeline with test questions
`init_state() / log_agent() / print_log()`	Shared state dict + structured logging

TRACKS

Choose Your Project Track

Both tracks teach the same core skills. Pick the one that excites you more.

Track A: Intelligence Hub

`hub_starter.py`

6 agents: Planner, Summarizer, Fact Extractor, Quiz Generator, Gap Analyzer, Critic

M1: Wire sequential pipeline
M2: Parallel with ThreadPoolExecutor
M3: Cost tracking + budget enforcement
M4: Evaluation harness (5 test Qs)
M5: Adaptive retry (Critic → Planner)

Track B: Debate System

`debate_starter.py`

7 agents: Planner, Researcher×2, Debater×2, Cross-Examiner×2, Judge, Synthesizer

M1: Wire sequential pipeline
M2: Parallel researchers + debaters
M3: Cross-examination round
M4: Cost tracking + budget decisions
M5: Tournament mode (3 topics)

Both tracks: agents are GIVEN (they work individually). YOUR job: build the pipeline that makes them work together

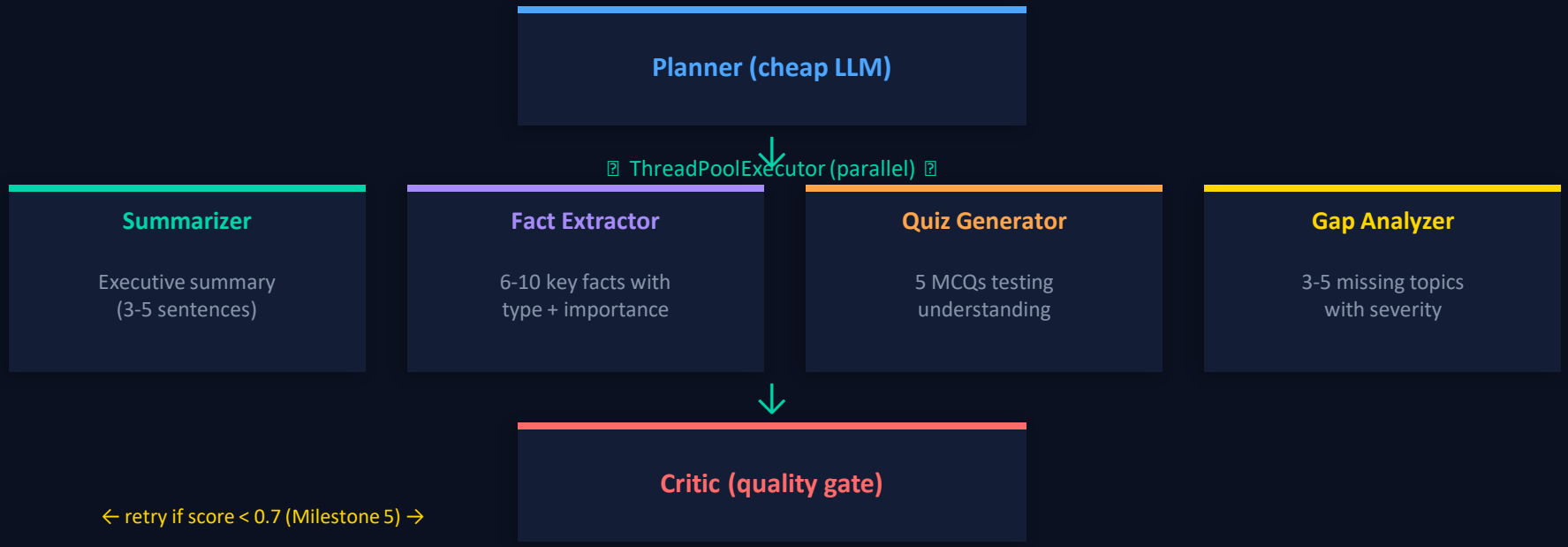
DAY 1

Day 1: Concepts + Milestones 1-2

9:00 – 10:30	**Concepts: GenAI → LLM → APIs → Agents → Multi-Agent**
10:30 – 11:00	**Environment Setup + run helpers.py tests**
11:00 – 11:15	**Choose Track + Read Starter Code**
11:15 – 12:00	**Milestone 1: Wire Sequential Pipeline (45 min)**
12:00 – 13:00	Lunch
13:00 – 13:45	**Milestone 1 continued / finish**
13:45 – 14:30	**Milestone 2: Parallel Execution (45 min)**
14:30 – 15:15	**Milestone 2 continued / testing**
15:15 – 16:00	**Start Milestone 3 (cost / cross-exam)**
16:00 – 16:30	**Day 1 Recap + Q&A**

TRACK A · HUB

Hub Architecture Overview



State flows: init_state() → planner fills 'plan' → agents fill summary/facts/quiz/gaps → critic validates all.

TRACK A · MILESTONE 1

Wire the Sequential Pipeline (45 min)

hub_starter.py — run_pipeline() function. Fill in the steps.

```
def run_pipeline(pdf_path, budget=0.50):
    tracker = CostTracker(budget=budget)

    # Step 1 - Load and index
    chunks = load_and_chunk(pdf_path)
    index = build_index(chunks)

    # Step 2 - Select diverse chunks
    state = init_state()
    for q in ["overview", "conclusion", ...]:
        state["chunks"].extend(
            search(index, chunks, q, k=4))
    # Deduplicate by chunk index!

    # Step 3-6 - Run agents SEQUENTIALLY
    state = planner(state, tracker)
    state = summarizer(state, tracker)
    state = fact_extractor(state, tracker)
    state = quiz_generator(state, tracker)
    state = gap_analyzer(state, tracker)
    state = critic(state, tracker)
```

Key Decisions

Chunk selection matters
Search for 'overview', 'conclusion', 'methodology' etc. to get diverse chunks

Deduplication
Same chunk may match multiple queries. Filter by chunk index.

Test after wiring
python hub_starter.py doc.pdf
All 6 agents should print output.

Done when: pipeline runs end-to-end. Critic prints a score. print_log() shows all agents.

TRACK A · MILESTONE 2

Parallel Execution (45 min)

Summarizer, Fact Extractor, Quiz Gen, Gap Analyzer are independent — run them in parallel.

```
from concurrent.futures import (
    ThreadPoolExecutor, as_completed
)

def run_parallel_agents(state, tracker):
    agents = {
        "summarizer": summarizer,
        "fact_extractor": fact_extractor,
        "quiz_generator": quiz_generator,
        "gap_analyzer": gap_analyzer
    }
    with ThreadPoolExecutor(max_workers=4) as ex:
        futures = {}
        for name, fn in agents.items():
            s_copy = {**state}  # OWN COPY!
            futures[ex.submit(fn, s_copy, tracker)] = name
        for future in as_completed(futures):
            name = futures[future]
            try:
                result = future.result()
```

Gotchas

Each agent needs OWN state copy
{**state} — otherwise race conditions

Merge results carefully
state["summary"] = result["summary"]
Don't forget to merge logs too

Error boundaries per agent
One failure shouldn't crash all 4. try/except around future.result()

Done when: pipeline is ~3x faster than M1. Time it: start = time.time() before and after.

TRACK B · DEBATE

Debate System Architecture

State keys: debate_plan, evidence_for/against, argument_for/against, judgment, cross_exam_*, synthesis

TRACK B · MILESTONE 1

Wire the Debate Pipeline (45 min)

debate_starter.py — run_debate() function. Fill in the 7 steps.

```
def run_debate(pdf_path, topic, budget=0.80):
    tracker = CostTracker(budget=budget)

    # Step 1 - Load + index + init state
    chunks = load_and_chunk(pdf_path)
    index = build_index(chunks)
    state = init_state(topic)
    state["topic"] = topic
    state["_index"] = index
    state["_all_chunks"] = chunks
    state["chunks"] = search(index, chunks, topic, k=8)

    # Step 2 - Planner
    state = debate_planner(state, tracker)

    # Step 3 - Researchers (sequential for M1)
    state = researcher(state, tracker, "for")
    state = researcher(state, tracker, "against")

    # Step 4 - Debaters (sequential for M1)
    state = debater(state, tracker, "for")
```

State Setup

state["topic"] must be set

state["_index"] and state["_all_chunks"] needed by researcher

state["chunks"] = initial context chunks from search

Test:
python debate_starter.py doc.pdf "Is Python better than Java for ML?"

Done when: all agents print output, judge declares a winner with scores.

TRACK B · MILESTONE 2

Parallel Execution (45 min)

Researchers are independent → run in parallel. Debaters too.

```
def run_parallel(fns_with_args, state, tracker):
    """Run [(fn, side), ...] in parallel."""
    results = {}
    with ThreadPoolExecutor(max_workers=2) as ex:
        futures = {}
        for fn, side in fns_with_args:
            s = {**state}  # copy!
            futures[ex.submit(fn, s, tracker, side)] = side
        for future in as_completed(futures):
            side = futures[future]
            result = future.result()
            results[side] = result
    # Merge: state["evidence_for"] = results["for"]["evidence_for"]
    return state

# Usage:
run_parallel([(researcher,"for"),(researcher,"against")],...)
```

Merge Strategy

No conflicts! Each side writes to different keys:

FOR → evidence_for, argument_for
AGAINST → evidence_against, argument_against

Just copy the relevant keys + merge logs.

Time it: should be ~2x faster.

Done when: pipeline runs ~2x faster. Both sides still get full evidence and arguments.

DAY 2

Day 2: Milestones 3-5 + Demos

9:00 – 9:30	**Day 1 Recap + Questions**
9:30 – 10:30	**Milestone 3 (Hub: Cost Tracking / Debate: Cross-Exam)**
10:30 – 11:30	**Milestone 4 (Hub: Eval Harness / Debate: Cost Tracking)**
11:30 – 12:00	**Milestone 4 continued / testing**
12:00 – 13:00	Lunch
13:00 – 14:30	**Milestone 5 (Hub: Adaptive Retry / Debate: Tournament)**
14:30 – 15:30	**Polish + prepare demo**
15:30 – 16:30	**Live Demos (5 min each)**
16:30 – 17:00	**Wrap-up + Next Steps**

MILESTONE 3

Milestone 3 — Both Tracks

Hub: Cost Tracking + Budget

1. Add tracker.check_budget() between stages
2. Catch RuntimeError gracefully
3. Print tracker.report() at end
4. Test: --budget 0.05 (should stop early)

Where to place checks?
After every agent? (safe but verbose)
After each stage? (practical)
Only at end? (too late!)

Debate: Cross-Examination

1. After judge(), check margin
2. If "razor-thin" or "narrow":
   → cross_examiner(state, tracker, "for")
   → cross_examiner(state, tracker, "against")
   → judge(state, tracker, round_num=2)
3. Run cross-examiners in PARALLEL
4. Budget check: skip round 2 if < $0.15

Engineering Q: Should budget override the margin check?

MILESTONE 4

Milestone 4 — Both Tracks

Hub: Evaluation Harness

1. Create EvalHarness with 5 test Qs
2. Each test: question + expected_keywords
3. Write pipeline_for_eval(question)
   that returns {answer, critic_score}
4. Run harness.run() + harness.report()

Target: >60% pass rate

Tip: Reuse FAISS index across tests
(don't rebuild for each question)

Debate: Cost Tracking + Budget

1. tracker.check_budget() between stages
2. Make round 2 decision budget-aware
   if tracker.remaining() < 0.15: skip
3. Print tracker.report() at end
4. Test: --budget 0.10
   Does it stop gracefully?

Cross-exam costs ~$0.08-0.10
(2 strong LLM calls + 1 judge)

MILESTONE 5

Milestone 5 — Both Tracks

Hub: Adaptive Retry

When Critic scores < 0.7:
1. state["_critic_feedback"] has hints
2. Re-run planner(state, tracker)
   (planner already reads feedback!)
3. Re-run parallel agents + critic
4. Track retry_count in state
5. Budget check before each retry

When NOT to retry:
Budget too low? Score = 0.0?
Already at max_retries?

Debate: Tournament Mode

Run 3 debates on same document:
1. Accept multiple topics
2. Reuse FAISS index (don't rebuild!)
3. Track per-topic: winner, scores, cost
4. Print tournament summary

Example output:
Topic 1: FOR wins (39/50 vs 34/50)
Topic 2: AGAINST wins (41/50 vs 37/50)
Topic 3: FOR wins (38/50 vs 36/50)
Overall: FOR 2-1, $0.45 total

REFERENCE

Mistakes Everyone Makes

✗ Forgetting {**state} in parallel	✓ Race conditions! Each thread needs its own state copy.
✗ Not merging logs from parallel agents	✓ state['log'].extend(result.get('log', [])) after merge.
✗ Silent except: pass in error boundaries	✓ Log errors to state['errors']. Critic will score missing output as 0.
✗ Infinite retry loop	✓ Always: while retry_count < max_retries. Check budget before retry.
✗ Not deduplicating chunks	✓ Same chunk matches multiple queries. Filter by chunk['index'].
✗ LLM JSON output not validated	✓ Wrap parse_json() in try/except. LLMs sometimes return garbage.
✗ Building everything before testing	✓ Test after EACH milestone. Bugs compound fast.

REFERENCE

Your Code Structure

```
workshop/
├── helpers.py           # ALL shared utilities
│   (load_and_chunk, build_index, search,
│    call_llm_cheap, call_llm_strong,
│    parse_json, CostTracker, EvalHarness,
│    SemanticCache, init_state, log_agent)
│
├── hub_starter.py       # Track A starter
├── debate_starter.py    # Track B starter
│
├── project_intelligence_hub.py  # Solution A
├── project_debate_system.py     # Solution B
│
├── _env.example         # Copy to .env
└── requirements.txt
```

What's Given vs Your Job

GIVEN (don't modify):

helpers.py — all building blocks
Agent functions — they work individually
CLI + report formatting

YOUR JOB (the interesting parts):

Pipeline wiring (run_pipeline / run_debate)
Parallel execution (run_parallel_agents)
Cost tracking integration
Eval harness / tournament mode
Adaptive retry logic

Solution files are provided — but try to solve each milestone yourself first!

DEMOS

5-Minute Demo Playbook

What to say, in order. Practise once before your slot.

0:00 – 0:40 — **Set the Scene**

"We chose Track A/B. Our document is [X]. Here's our pipeline architecture."

0:40 – 2:00 — **Live Run 1**

Run the pipeline. Show output from each agent. Point to the cost report.

2:00 – 3:20 — **Live Run 2 / Edge Case**

Show parallel execution speed. Trigger a retry/cross-exam. Show logs.

3:20 – 4:30 — **Highest Milestone**

Show the most advanced milestone you completed. Explain what it does.

4:30 – 5:00 — **Honest Reflection**

"If we had 2 more hours, we'd improve [X]. Currently struggles with [Z]."

If something breaks during demo, explain what broke and why. That shows you understand the system.

EVALUATION

How You'll Be Evaluated

35% **Pipeline Works**
Runs end-to-end. All agents produce output. No crashes.

25% **Milestones**
How many of the 5 milestones did you complete?

20% **Parallel + Quality**
Parallel execution works. Critic/Judge actually validates.

10% **Code & Logs**
Error boundaries. State log shows what each agent did.

10% **Demo & Reflection**
Can explain decisions. Honest about what broke and why.

Working simple > broken complex. 3 milestones working perfectly beats 5 milestones half-done.

REFERENCE

Quick Reference — State Schemas

Hub State (hub_starter.py)

```
state = {
  "query": "",
  "chunks": [{text, score, index}],
  "plan": {document_type, main_topic,
           key_themes, ...},
  "summary": {summary, key_points},
  "facts": {facts: [{fact, type, importance}]},
  "quiz": {questions: [{question, options,
           correct, explanation}]},
  "gaps": {gaps: [{topic, why_important,
           severity}], coverage_score},
  "critic": {overall_score, scores, verdict},
  "critic_score": 0.85,
  "_critic_feedback": "hints for retry",
  "log": [],
  "errors": []
}
```

Debate State (debate_starter.py)

```
state = {
  "topic": "Is Python better for ML?",
  "_index": faiss_index,
  "_all_chunks": chunks,
  "chunks": [{text, score, index}],
  "debate_plan": {topic_restated,
    for_position, against_position,
    dimensions},
  "evidence_for": [{text, score, index}],
  "evidence_against": [...],
  "argument_for": {opening_statement,
    arguments: [{point, evidence, reasoning}],
    counter_to_opposition},
  "argument_against": {...},
  "judgment": {for_score, against_score,
    winner, margin, reasoning},
  "cross_exam_for": {...},
  "synthesis": {balanced_analysis, ...},
  "log": [], "errors": []
}
```

REFERENCE

CostTracker & Model Pricing

```
# Usage (already wired into agent functions):
tracker = CostTracker(budget=0.50)

result = call_llm_cheap(...)   # auto-picks model
tracker.record(result, "planner")

tracker.remaining()     # dollars left
tracker.check_budget()  # raises if over
tracker.report()        # prints breakdown
```

Model Pricing (per 1M tokens)

GPT-4o-mini: $0.15 in / $0.60 out
GPT-4o: $2.50 in / $10.00 out
Claude Haiku: $0.80 in / $4.00 out
Claude Sonnet: $3.00 in / $15.00 out
Embeddings: $0.02 in

Use call_llm_cheap() for Planner (routing). Use call_llm_strong() for Reasoner/Debater/Critic (quality).

REFERENCE

When It Breaks — Debug Systematically

Agent returns empty / wrong output	→ print(state["chunks"][:2]) — are the chunks relevant?
Critic always returns score = 1.0	→ Critic prompt says "Be STRICT" — but check: are you passing real outputs?
JSON decode error from LLM	→ parse_json() handles this. If still failing: check LLM prompt says 'Return ONLY JSON'
Pipeline runs forever (retry loop)	→ Check: while retry_count < max_retries. Print retry_count in loop.
Parallel agents: missing output	→ Check: did you merge result_state[key] back into main state?
KeyError on state access	→ print(state.keys()) — is the agent that should fill this key actually running?
Budget exceeded unexpectedly	→ tracker.report() shows per-agent cost. Is one agent too expensive?

NEXT STEPS

After This Workshop

This Week

Run your pipeline on 3 different documents — see where it breaks. Add a Streamlit UI (10 lines of code). Read Anthropic docs on tool use: docs.anthropic.com

This Month

Build a proper eval harness: 20 Q&A pairs with ground truth. Try a real use case: textbook chapter, project docs. Compare LLMs: same pipeline, different models — which gives better scores?

Beyond

Explore autonomous coding agents (SWE-Agent, Claude Code). Multi-modal agents (images + tables). Build something people actually use.

You Built a Multi-Agent AI System.

Most people only talk about this stuff.

- You understand WHY the architecture works, not just how
- You have working code you can show in interviews
- You know the real problems: hallucinations, cost, parallelism

Now go build something interesting.